

Projektdokumentation: KI-Fitness Coach

Autoren: Fabian Wild & Arwin Faethe

Fach: Data Science & Artificial Intelligence (DSAI)

1 Einleitung & Zielsetzung

Der KI-Fitness Coach ist ein KI-Modell, das in Echtzeit die Ausführung von Fitnessübungen (wie z.B. Hampelmänner) über eine Webcam analysiert und bewertet. Ziel des Projekts ist es, Nutzern direktes Feedback zu geben, ob eine Übung korrekt ausgeführt wurde ("Good") oder ob Fehler in der Haltung vorliegen ("Arm_Error", "Leg_Error"). Das System zählt dabei nicht nur die Wiederholungen, sondern agiert als intelligenter Trainer, der die Bewegungsqualität überwacht.

2 Technologie-Stack

Für die Umsetzung wurden folgende Haupttechnologien verwendet:

- **Python:** Hauptprogrammiersprache.
- **MediaPipe (Google):** Für das Pose-Tracking (Skeletterkennung). MediaPipe wurde OpenPose vorgezogen, da es hochoptimiert für Standard-CPU's ist (30+ fps), 3D-Koordinaten liefert und wesentlich ressourcenschonender arbeitet.
- **OpenCV (cv2):** Für die Kameraansteuerung, Bildverarbeitung und das Zeichnen der Benutzeroberfläche.
- **Scikit-Learn:** Für das Machine Learning Modell (*RandomForestClassifier*).
- **Pandas & NumPy:** Für die Datenverarbeitung, Matrixoperationen und das Speichern der Datensätze.

3 Datenerhebung (training.py)

Da bestehende Datensätze (wie das *Mocap 6* oder *Aerobic-movement-skeleton* Dataset) Probleme aufwiesen, wurde ein eigenes Datenset erstellt.

3.1 Aufnahmeprozess

Das Skript *training.py* dient als Data Collector. Es nutzt MediaPipe, um in Echtzeit vier Kernwinkel des Körpers zu berechnen:

- Linke Schulter
- Rechte Schulter

- Linke Hüfte
- Rechte Hüfte

Logik für eine Wiederholung:

- **Start:** Hebt der Nutzer die Arme über 110 Grad, beginnt die Aufzeichnung.
- **Ende:** Senkt der Nutzer die Arme wieder unter 85 Grad, wird die Aufzeichnung gestoppt.
- **Klassifizierung:** Nach dem Sprung friert das Bild ein und der Mensch klassifiziert die Daten manuell per Tastendruck: [G] Good, [A] Arm_Error, [B] Leg_Error oder [X] Verwerfen.

3.2 Datenvereinheitlichung (Resampling)

Da nicht jeder Sprung exakt gleich lang dauert (unterschiedliche Anzahl an Bildern), werden die gesammelten Bilder einer Bewegung auf exakt 30 Bilder gestaucht oder gestreckt (Interpolation). Bei 4 Winkeln pro Bild ergeben sich so exakt 120 Datenpunkte pro Sprung.

4 Datenvorverarbeitung

4.1 EMA-Filter (Signalglättung)

Rohe Knotenpunkte der Webcam-Erkennung zittern oft. Um sprunghafte, falsche Winkelberechnungen zu vermeiden, wurde ein Exponential Moving Average (EMA) Filter (mit $\alpha = 0.4$) implementiert. Dieser glättet die Bewegungskurven in Echtzeit, filtert Ausreißer heraus und erhält die Live-Reaktionszeit.

4.2 Data Augmentation (Datenerweiterung)

Der Datensatz bestand aus 150 Sprüngen. Ein so kleiner Datensatz neigt zum Auswendiglernen (Overfitting).

Lösung: Im Trainingsprozess (main.py) werden die Trainingsdaten verfünffacht. Dabei wird den Original-Winkeln ein leichtes Rauschen (Gaußsches Rauschen) hinzugefügt und die Werte minimal skaliert (gestreckt/gestaucht). Dadurch lernt das Modell robustere Muster.

5 Das Machine Learning Modell

Als Klassifikator wurde ein **Random Forest Algorithmus** gewählt.

5.1 Funktionsweise (Ensemble Prinzip)

Ein Random Forest besteht aus vielen einzelnen Entscheidungsbäumen. Jeder Baum wird mit einer zufälligen Stichprobe der Daten (Bootstrap Sampling) trainiert. Bei einer Vorhersage stimmen alle Bäume ab (Majority Voting).

5.2 Vorteile für dieses Projekt

- Sehr robust gegenüber Ausreißern.
- Hohe Genauigkeit bei komplexen Daten.
- Geringere Neigung zu Overfitting durch das Zusammenwirken vieler schwacher Lerner.

5.3 Trainingsparameter

- `n_estimators`: 300 (Anzahl der Bäume)
- `max_depth`: 8 (Maximale Tiefe der Entscheidungsbäume, um Overfitting vorzubeugen)

Das Modell trainiert in einer Schleife (bis zu 30 Versuche) mit einem 80/20 Train-Test-Split, bis eine Genauigkeit von mindestens 90% (später 100%) erreicht wird. Das fertige Modell wird als *fitness_model.pkl* gespeichert. Am Ende wurde eine Genauigkeit von 100% erreicht.

6 Die Hauptanwendung (main.py)

Das Skript main.py vereint das trainierte Modell mit der Live-Webcam.

6.1 Ablauf

- **Start:** Das Skript prüft, ob ein trainiertes Modell existiert. Wenn nicht, wird es automatisch mit dem CSV-Datensatz trainiert.
- **Tracking:** MediaPipe erfasst das Skelett, der EMA-Filter glättet die Winkel.
- **Bewegungserkennung:** Die gleiche Logik wie im Collector (110° Auf, 85° Ab) erkennt einen vollständigen Sprung.
- **Bewertung:** Die 30 interpolierten Frames des Sprungs werden an den Random Forest übergeben. Dieser gibt die Diagnose zurück (Good, Arm_Error, Leg_Error).

6.2 Benutzeroberfläche (UI)

Die Applikation bietet ein intuitives Overlay:

- **REPS:** Gesamtzahl der absolvierten Wiederholungen.
- **GOOD:** Zähler für Übungen, die von der KI als "Good" eingestuft wurden.
- **STATE:** Zeigt an, ob sich der Nutzer in der Aufwärts- (UP) oder Abwärtsbewegung (DOWN) befindet.
- **KI DIAGNOSE:** Zeigt den genauen Fehler an.
- **Visuelles Feedback:** Das von MediaPipe gezeichnete Skelett färbt sich Grün bei einer korrekten Ausführung und Rot bei einem Fehler.